DUMMY INTERCOM · DEBUG

# デバッグ手順

エミュレーション / シミュレーション / 実機。受信専用インカムブリッジ（ESP32-WROOM-32 / ESP-IDF v5.4 / Bluedroid HFP-AG）の不具合を、無線を使わないロジック検証から HCI レベルのプロトコル観測まで、層ごとに切り分けて追うための手順書。

---

## 0 レイヤと使い分け

「どこが怪しいか」で道具が変わる。上の層ほど手軽・非侵襲、下の層ほど実機依存。

| | |
|---|---|
| **層1<br>ロジック** | 無線なしでコードだけ検証。 QEMU エミュレーション、Wokwi シミュレーション、ホスト（Linux ターゲット）ユニットテスト。ブート順・NVS・LED 状態機械・リングバッファ / ジッタバッファ / RMS / CoD の算術。Bluetooth は動かせない。 |
| **層2<br>実機 + 観測** | 実機を止めずに覗く。 ログ（`ESP_LOGx`）、`idf.py monitor` のバックトレース自動デコード、パニック GDB スタブ、フラッシュへのコアダンプ。BT 接続・SCO のふるまいはここで追う。 |
| **層3<br>侵襲的** | 止めて中を見る。 JTAG + OpenOCD + GDB でステップ実行・ウォッチポイント。BT 接続処理には不向き（CPU を止めるとコントローラがデッドライン割れ）。BT 確立後のアプリ側ロジック限定。 |
| **横断<br>プロトコル** | 相手（**SRL-3**）との通信を見る。 Linux PC + USB BT ドングル + `btmon`。SLC / `AT+BAC` / `+BCS` / SCO・コーデックのネゴを air レベルで観測。最重要リスク（T3 / ISS-008）はこれで切り分ける。 |

## 1 症状 → 手法の早見表

| 症状 / 調べたいこと | まず使う手法 | 章 |
|---|---|---|
| 起動時にクラッシュ / 再起動ループ | monitor バックトレース → コアダンプ | §5 §6 |
| NVS / パーティション / `storage.c` の挙動 | QEMU（実機不要） | §2 |
| LED 3 状態・BOND-CLR ボタン | Wokwi、または QEMU + GDB | §3 §2 |
| ジッタバッファ / RMS / CoD の計算違い | ホストユニットテスト | §4 |
| SLC は張れるが SCO が張れない（T3 / ISS-008） | `btmon` + HCI ログ + monitor | §9 §5 |
| SCO は張れるが音が出ない / 途切れる | ログ（RMS / underrun）+ `esp_timer` 計測 | §5 §10 |
| `recv_cb` が重くて BT 全体が破綻（ISS-013） | コード確認 + `esp_timer` + Int WDT ログ | §10 |
| 30 分連続でオーバー / アンダーラン（ISS-015） | 長時間ログ + underrun カウンタ | §5 §10 |
| I2S タスク・リングバッファのロジック | JTAG ステップ実行（BT 確立後） | §8 |
| メモリ破壊 / スタックオーバーフロー | ヒープポイズニング + ウォッチポイント | §10 |

## 2 エミュレーション — QEMU

ESP-IDF は Espressif 版 QEMU で ESP32 イメージをそのまま実行できる。**JTAG** ハードなしで **GDB** ステップ実行が可能なのが最大の利点。

### 導入（このマシンは未導入）

```
# qemu-xtensa をツールとして追加
python $IDF_PATH/tools/idf_tools.py install qemu-xtensa
# もしくは install.sh をやり直すとき: ./install.sh --enable-qemu esp32
. $HOME/esp/esp-idf/export.sh   # PATH 再読込
```

### 実行

```
idf.py build
idf.py qemu                    # エミュレータ起動（シリアルが端末に出る）
idf.py qemu monitor            # monitor 付きで起動
idf.py qemu gdb                # GDB アタッチ状態で起動（別端末不要）
```

GDB 側：`b storage.c:35` → `c` → ブレーク → `n` / `s` / `p key` / `bt`。

> **QEMU の限界 — BLUETOOTH コントローラが無い**
>
> QEMU-ESP32 は BT/Wi-Fi 無線を実装していない。`esp_bt_controller_init()` 以降は正常動作しない。使いどころは **BT** 初期化より手前（`app_main.c` の NVS 初期化、`storage.c`、パーティション、`app_config.h` 定数、`leds.c` の GPIO ロジック）に限定する。
>
> BT 抜きで層1 検証をしたいなら、`app_main.c` に `#ifndef DEBUG_NO_BT` ガードを入れた「ロジック専用ビルド」を用意すると QEMU で全経路を歩ける。

I2S は DMA レジスタ動作までは回るが実音は出ない。波形の確認は Wokwi か実機で。

---

## 3 シミュレーション — WOKWI

ビルド済みファームを仮想 ESP32 + 周辺部品で動かす。GPIO / LED / ボタン / 起動ログの目視に向く。VS Code 拡張「Wokwi for VS Code」または `wokwi-cli`。

### プロジェクトに置くファイル

```
# wokwi.toml
[wokwi]
version = 1
firmware = "build/dummy_intercom.bin"
elf      = "build/dummy_intercom.elf"
```

```
# diagram.json（抜粋: ESP32 + LED×3 + タクト）
{
  "version": 1,
  "parts": [
    { "type": "board-esp32-devkit-c-v4", "id": "esp" },
    { "type": "wokwi-led", "id": "pwr",  "attrs": { "color": "green" } },
    { "type": "wokwi-led", "id": "link", "attrs": { "color": "blue" } },
    { "type": "wokwi-led", "id": "sco",  "attrs": { "color": "yellow" } },
    { "type": "wokwi-pushbutton", "id": "bond", "attrs": { "bounce": "1" } }
  ],
  "connections": [
    [ "esp:32", "pwr:A",  "green", [] ],
    [ "esp:33", "link:A", "blue",  [] ],
    [ "esp:13", "sco:A",  "yellow",[] ],
    [ "esp:14", "bond:1.l", "green", [] ],
    [ "pwr:C","esp:GND.1","black",[] ], [ "link:C","esp:GND.1","black",[] ],
    [ "sco:C","esp:GND.1","black",[] ], [ "bond:2.l","esp:GND.1","black",[] ]
  ]
}
```

> **WOKWI の限界**
>
> Classic Bluetooth / HFP / SCO は非対応（BLE は一部のみ）。PCM5102A の部品も無い。`leds.c` の状態遷移、BOND-CLR 長押し（3 秒）、起動シーケンスの確認までが実用範囲。BT が絡む経路は「初期化ログが出るところまで」。

---

### 4 シミュレーション – ホストユニットテスト（LINUX ターゲット）

純粋な計算ロジックは PC 上で高速に回せる。`idf.py --preview set-target linux` + Unity / CMock。BT・I2S ドライバは Linux ビルドできないので、対象を純関数に切り出してテストコンポーネントから叩く。

### このコードでホストテストに向く部分

| 対象 | 場所 | 検証内容 |
|---|---|---|
| ジッタバッファ閾値 | app_config.h / audio_i2s.c | prefill / target / RB サイズの整合、ms 換算 |
| RMS 計算 | bt_data_path.c | 既知 PCM 列 → 期待 RMS、桁溢れ |
| CoD 値 | app_main.c | service\|major\|minor → 0x60020C |
| CIND / COPS / CLCC 応答生成 | bt_hf_ag.c | 各クエリに対する固定応答文字列 |
| L=R 複製 / ゼロ埋め | audio_i2s.c / bt_data_path.c | モノ→ステレオフレーム、send の sz 厳守 |

```
# 例: テスト専用ミニプロジェクトを作る
mkdir -p test_host/main && cd test_host
# main/ に対象の純関数 .c と test_*.c (Unity) を置く
idf.py --preview set-target linux
idf.py build && ./build/test_host.elf
```

オフライン再生テスト：実機で `idf.py monitor` 中に録れた（あるいは録音した）CVSD/PCM を配列にして data path 関数へ流し、リングバッファ充填とアンダーラン挙動をホストで再現する。ISS-015（クロックドリフト）の当たりを付けるのに有効。

---

## 5 実機 – ログ

BT/SCO のタイミング依存コードは「止めると壊れる」。層2 の基本はログ。本ファームは既に `ESP_LOGx` が充実している。

| TAG | 役割 |
|---|---|
| app_main | NVS / コントローラ / Bluedroid 起動、自機 BDA |
| bt_hf_ag | SLC / SCO / DIAL / UNAT / codec ネゴ / 再接続 |
| bt_data_path | recv/send コールバック、RMS（1 秒毎） |
| audio_i2s | I2S TX タスク、ジッタバッファ、アンダーラン |
| storage | ピア BDA の保存 / 読出 / 消去 |
| leds | PWR / LINK / SCO 駆動、BOND-CLR 監視 |

```
# 実行時に特定タグだけ詳細度を上げる（コードに1行）
esp_log_level_set("bt_hf_ag", ESP_LOG_DEBUG);

# monitor 側で絞り込み表示
idf.py -p /dev/ttyUSB0 monitor --print-filter "bt_hf_ag:D bt_data_path:D *:I"
```

> **RECV_CB にログを足さない（ISS-013）**
>
> `recv_cb` は BTC タスク文脈。`ESP_LOGx` / `malloc` / `i2s_channel_write` をここで呼ぶと SCO 全体が破綻する。観測は「1 秒集計済みの RMS / カウンタ」を別タスクで出す形にとどめる。生データを見たいときはリングバッファ経由で別タスクへ。

---

## 6 実機 – MONITOR & パニック

- `idf.py monitor` はパニック時のバックトレースを自動でアドレス→`関数:行` に変換。
- パニック時 **GDB** スタブ（JTAG 不要）：`menuconfig` → `CONFIG_ESP_SYSTEM_PANIC_GDBSTUB=y`。クラッシュすると UART で GDB を待つ。

  ```
  xtensa-esp32-elf-gdb build/dummy_intercom.elf
  (gdb) target remote /dev/ttyUSB0
  (gdb) bt full        # クラッシュ地点のスタックと変数
  ```

- Task WDT / Interrupt WDT のログは「どのタスクが CPU を離さなかったか」を名指しする。`recv_cb` 起因なら Int WDT に `BTC` / SCO 系タスクが出る。
- `ESP_ERROR_CHECK()` 失敗・`assert`・`abort()` はファイル:行がそのまま出る。`esp_hf_ag_*` の戻り値は握りつぶさずログへ。

---

## 7 実機 – コアダンプ

クラッシュ時の全タスク状態をフラッシュに保存し、後から GDB で解析する。再現性が低いバグ向け。

```
# menuconfig
CONFIG_ESP_COREDUMP_ENABLE_TO_FLASH=y
CONFIG_ESP_COREDUMP_DATA_FORMAT_ELF=y
CONFIG_ESP_COREDUMP_CHECKSUM_CRC32=y
```

```
idf.py coredump-info      # サマリ（パニック理由・タスク一覧）
idf.py coredump-debug     # GDB を core に対して起動（bt / p / info threads）
# 保存済みコアダンプを消す
python $IDF_PATH/components/espcoredump/espcoredump.py erase -p /dev/ttyUSB0
```

---

### 8 実機 – JTAG ステップ実行

アプリ側ロジック（I2S タスク、リングバッファ、状態機械）を単発で追うとき。**BT** 接続処理そのものには使えない。

#### 必要なもの

Espressif **ESP-Prog**（推奨）または FT2232H ボード。ELEGOO の CP2102 は JTAG 非対応、DevKit も JTAG 端子を出していないので外付け必須。OpenOCD（`v0.12.0-esp32`）は導入済み。

| プローブ | ESP32 | 備考 |
|---|---|---|
| TMS | GPIO14 | = BOND-CLR ボタン（競合） |
| TCK | GPIO13 | = SCO LED（競合） |
| TDO | GPIO15 | ストラッピング |
| TDI | GPIO12 | ストラッピング（フラッシュ電圧） |
| GND | GND | — |

> **ピン競合 – デバッグ中は LED/ボタンを外す**
> GPIO13（SCO LED）と GPIO14（BOND-CLR）は JTAG と衝突する。デバッグセッション中は `leds.c` の初期化をコンパイルスイッチで無効化するか、これらの配線を物理的に外す。GPIO12 はストラッピングなので、JTAG プローブ接続時のレベルに注意（多くの ESP-Prog は問題ないが、起動失敗時は eFuse / フラッシュ電圧設定を確認）。

#### デバッグビルド設定（menuconfig）

```
CONFIG_COMPILER_OPTIMIZATION_DEBUG=y        # -Og（-Os だと変数が消える）
CONFIG_ESP_TASK_WDT_EN=n                    # ブレークで発火するWDTを止める
CONFIG_ESP_INT_WDT=n
CONFIG_FREERTOS_WATCHPOINT_END_OF_STACK=y   # スタック溢れをHWで捕捉
```

**起動**

```
idf.py openocd                          # 端末1（ESP-Prog を自動認識）
idf.py gdb                               # 端末2（CLI）
idf.py gdbgui                            # もしくはブラウザGUI
# VS Code なら拡張「Espressif IDF」でブレークポイント＋F10/F11
```

```
(gdb) b audio_i2s.c:  # I2S TX タスクの処理単位
(gdb) mon reset halt
(gdb) c
(gdb) n / s / finish
(gdb) watch s_underruns      # HW ウォッチポイント（ESP32 は 2 個）
(gdb) p *rb                  # リングバッファ構造体
```

> **BLUETOOTH のリアルタイム制約**
> ブレークで CPU を止めると BT コントローラがデッドラインを割り、SLC / SCO が切断される。**SCO** 確立後のアプリ側ロジックを対象にし、条件付きブレーク（`break ... if`）や `watch` でヒット回数を絞る。BT 確立フェーズを追うのは §9（btmon）の仕事。

ハードウェアウォッチポイント（**2** 個）はバッファ破壊の検出に強力。「いつこの値が書き換わったか」を `watch` で捕まえる。

---

## 9 横断 — BLUETOOTH / HCI レベル観測

最重要リスク T3（SCO 確立）/ ISS-008（codec ネゴハング）は、ダミーインカム機側だけ見ても分からない。**SRL-3** との **air** 上のやり取りを見る。

### 方法A：別 Linux PC + USB BT ドングル + btmon（仕様 §5.4 / 14.2）

```
sudo btmon                              # HCI トレース（別端末で hcitool / bluetoothctl 操作）
# もしくは Wireshark で bluetooth-monitor インターフェース
```

見るポイント：

- `BRSF`（feature 交換）、`AT+BAC`（HF が提示するコーデック一覧）
- `+BCS: 1`（CVSD）か `+BCS: 2`（mSBC）、その後の `AT+BCS` 応答
- `HCI Setup Synchronous Connection` → `Synchronous Connection Complete`（成功 / status）
- SCO パケットが流れ始めるか、ネゴ後に無音イベントで止まっていないか

この構成は「ダミーインカム機 ↔ SRL-3」の通信を第三の受信機で盗聴するのではなく、**PC** 自身を **AG** 役にして **SRL-3** と繋ぎ、同じ **SLC/SCO** シーケンスを **btmon** で観測して基準挙動を得る、という切り分けにも使える。

### 方法B：ESP32 単体の HCI トレース（ドングル不要）

```
CONFIG_BT_HCI_LOG_DEBUG_EN=y        # HCI 入出力をメモリバッファに保持
```

アサート時やコマンドで直近の HCI パケットをダンプできる。btmon ほど見やすくないが、別 PC が用意できないときの代替。

---

## 10 メモリ & タイミング

### メモリ破壊

```
CONFIG_HEAP_POISONING_COMPREHENSIVE=y       # 前後マーカー + 解放後チェック
// コード中の疑わしい箇所で
heap_caps_check_integrity_all(true);
// リーク調査
heap_trace_start(HEAP_TRACE_LEAKS); ... heap_trace_stop(); heap_trace_dump();
```

### スタック

```
uxTaskGetStackHighWaterMark(NULL)    // 各タスクの余白。I2S TX / BtAppT を監視
```

### タイミング（ISS-015 クロックドリフト、T5 遅延）

- `esp_timer_get_time()` で `recv_cb` の到着間隔と I2S 書き込み周期を記録 → 8 kHz 名目に対する実レートのずれを算出
- アンダーラン / オーバーラン回数をカウンタ化し 1 秒毎にログ（`audio_i2s`）
- 空き GPIO を `recv_cb` と I2S 書き込みでトグル → ロジックアナライザ / オシロで周期と位相を可視化（T5 の遅延測定もこれ）
- SystemView（要 JTAG）でタスクスケジューリングと割込みを時間軸表示

## 11 このプロジェクトの未解決項目への当てはめ

| ISS / リスク | 内容 | 推奨手法 |
|---|---|---|
| ISS-008 / 012 | SCO codec ネゴがハング（T3） | §9 btmon + §9B HCI ログ + §5 monitor。**JTAG** 不可 |
| ISS-013 | recv_cb が重く BT 破綻 | コード grep 確認 + §10 esp_timer 計測 + Int WDT ログ |
| ISS-014 | timer cb から `esp_hf_ag_*` 直呼び | コード確認（全て dispatch 経由か）+ §5 ログ |
| ISS-015 | I2S / SCO クロック独立ドリフト（30 分） | §4 オフライン再生 + §10 esp_timer + underrun カウンタ長時間ログ |
| ISS-016 | 通話偽装（call=1）の副作用 | §5 monitor（SRL-3 の反応 AT）+ §9 btmon（実機） |
| ISS-017 | PCM5102A 8 kHz 再構成の音質 | 実機のみ（別紙 wiring）。§10 GPIO トグルで I2S 周期確認 |
| ブート / NVS / storage | ロジック | §2 QEMU |
| LED 状態機械 / BOND-CLR | ロジック | §3 Wokwi / §2 QEMU+GDB |
| ジッタバッファ / RMS / CoD | 算術 | §4 ホストユニットテスト |

## 12 コマンド早見表

```
# --- 層1: エミュ / シム ---
python $IDF_PATH/tools/idf_tools.py install qemu-xtensa
idf.py qemu gdb                          # QEMU + GDB（JTAG不要）
idf.py --preview set-target linux && idf.py build   # ホストテスト

# --- 層2: 実機観測 ---
idf.py -p /dev/ttyUSB0 monitor --print-filter "bt_hf_ag:D *:I"
idf.py coredump-info ; idf.py coredump-debug

# --- 層3: JTAG ---
idf.py openocd            # 端末1
idf.py gdb                # 端末2  /  idf.py gdbgui

# --- 横断: HCI ---
sudo btmon                # 別 Linux PC + USB BT ドングル
```

---

参照: `dummy_intercom_spec.md` rev 0.6（§5.4 プロトコル観測 / §11 テスト計画 / §14.2 開発環境）、`docs/agent_team/issue_list.md`（ISS-008〜017）、`docs/agent_team/test_report.md`。 ログ TAG は `main/*.c` の実装に準拠。ビルド / 書き込みは別紙 `esp32-flashing-guide.pdf`、配線は `esp32-pcm5102a-wiring.pdf`。 ESP-IDF v5.4 / OpenOCD v0.12.0-esp32 は導入済み、QEMU（qemu-xtensa）は要追加インストール。